

깃허브 사용 보고서

목 차	1. Git 브랜치 전략 2. 브랜치 보호 설정 3. 사용 방법
1. Git 브랜치	
[브랜치 전략] 프로젝트에서 안정적인 코드 관리를 위해 Git Flow 전략을 적용하며, 주요 브랜치는 다음과 같이 운영됨	
- main 브랜치 - 항상 안정적인 코드만 포함하며, 프로덕션 배포를 위한 최종 코드가 저장 - 직접 수정이 불가능하며, PR(Pull Request)을 통해서만 변경이 - develop 브랜치 - 새로운 기능 개발 및 테스트가 진행되는 기본 브랜치 - 기능 개발이 완료된 후 main 브랜치로 병합 - feature 브랜치(예: su-feature-login) - 개별 기능을 개발하기 위한 브랜치 - 기능 개발 후 develop 브랜치에 PR을 요청하여 병합 - develop 브랜치에서 최종 테스트 후 main 브랜치로 PR을 올려 배포 준비를 함	
2. 브랜치 보호 설정	
[브랜치 보호 설정] 코드 품질과 안정성을 보장하기 위해 main 및 develop 브랜치에 대해 브랜치 보호(Branch Protection Rules)를 적용함	
[설정 경로] Repository → Settings → Branches → Add classic branch protection rule	
[설정 항목] - Branch name pattern: main, develop (각각 설정) - Require a pull request before merging - PR 없이 직접 병합을 방지 - 최소 승인 인원 지정(예: 1명 이상) - Require status checks to pass before merging	

- 빌드 및 테스트 자동화 (GitHub Actions 활용)
- Status check이 실패하면 병합 불가
- Require branches to be up to date before merging
 - 최신 코드 기준으로 병합하도록 강제
- Do not allow bypassing the above settings
 - 위 설정을 우회하는 모든 방법 차단

[브랜치 보호 정책]

- main 및 develop 브랜치에 직접 수정 금지
- 새로운 브랜치를 생성하여 개발 후 PR을 통해서만 병합 가능
- PR 요청 시, 즉시 merge 불가
 - 최소 1명 이상의 승인(review)이 필요
 - Status check(자동 빌드 및 테스트) 통과 필수
- 팀원 수가 증가할수록 main 브랜치 보호의 중요성 증가
 - 이를 위해 철저한 브랜치 분리 및 코드 검토 과정 유지

3. 사용 방법

[Git 협업 및 PR 프로세스]

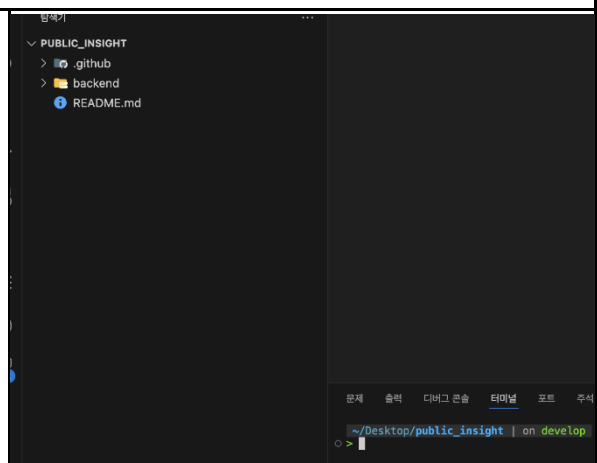
프로젝트에서는 su와 ha 두 명의 사용자가 협업하며, 새로운 기능을 개발하고 배포하기 위해 체계적인 Pull Request(PR) 및 코드 리뷰 절차를 따른다

[1. 기능 개발 (Feature 브랜치 작업)]

su가 새로운 기능을 개발하기 위해 develop 브랜치를 최신 상태로 가져온 후, 새로운 기능 브랜치를 생성하고 작업을 진행함

1. develop 브랜치 최신화

- git checkout develop
- git pull origin develop



2. 새로운 기능 브랜치 생성 - <code>git checkout -b su-feature-login</code>	문제 출력 디버그 콘솔 터미널 포트 주석 <pre>~/Desktop/public_insight on develop > git checkout -b su-feature-login 새로 만든 'su-feature-login' 브랜치로 전환합니다 ~/Desktop/public_insight on su-feature-login > </pre>
3. 파일 수정 후 커밋 - <code>git add.</code> - <code>git commit -m "로그인 기능 추가"</code>	<pre>> git add . ~/Desktop/public_insight on su-feature-login +1 > git commit -m "로그인 기능"</pre>
4. 원격 저장소에 브랜치 푸시 - <code>git push --set-upstream origin su-feature-login</code>	<pre>> git push --set-upstream origin su-feature-login 오브젝트 나열하는 중: 4, 완료. 오브젝트 개수 세는 중: 100% (4/4), 완료. Delta compression using up to 8 threads 오브젝트 압축하는 중: 100% (3/3), 완료. 오브젝트 쓰는 중: 100% (3/3), 539 bytes 539.00 KiB/s, 완료. Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0) remote: Create pull request for 'su-feature-login' on GitHub by visiting: remote: https://github.com/punggiboy/public_insight/pull/new/su-feature-login remote: To github.com:punggiboy/public_insight.git * [new branch] su-feature-login -> su-feature-login branch 'su-feature-login' set up to track 'origin/su-feature-login'. ~/Desktop/public_insight on su-feature-login > </pre>

[2. Develop 브랜치로 PR 요청 및 코드 리뷰]

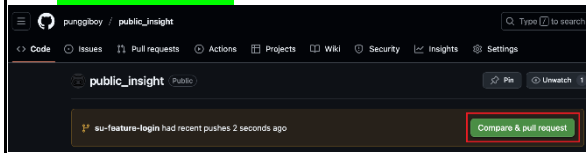
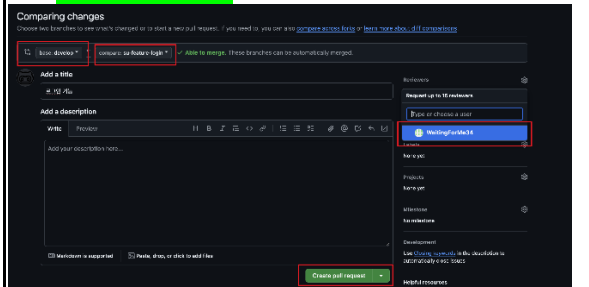
Pull Request(PR) 생성

- su가 GitHub에 접속하면 PR을 생성하라는 메시지가 표시됨
- PR을 생성할 때, 병합 대상 브랜치를 develop으로 설정
 - `develop <- su-feature-login`
- 코드 리뷰를 맡을 팀원(예: ha)을 지정

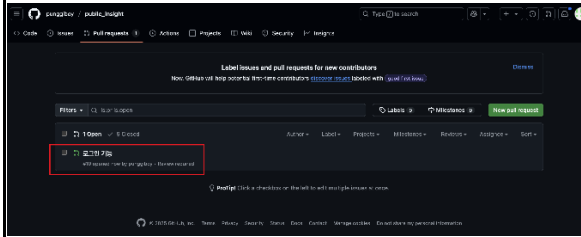
코드 리뷰 및 승인

- ha가 코드 내용을 확인하고 리뷰 및 승인(Approve)함
- 코드가 적절하면 su-feature-login 브랜치는 develop에 병합
- su-feature-login 브랜치 삭제

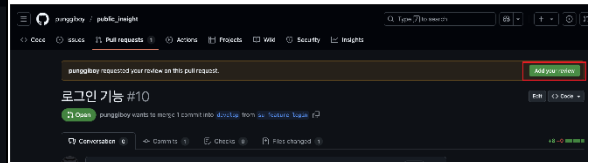
*빨간 네모로 볼 것

(1) su의 깃허브 	(2) su의 깃허브  *WaitingForMe34를 ha로 볼 것
--	---

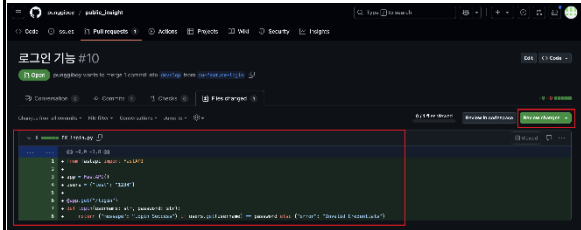
(3)ha의 깃허브



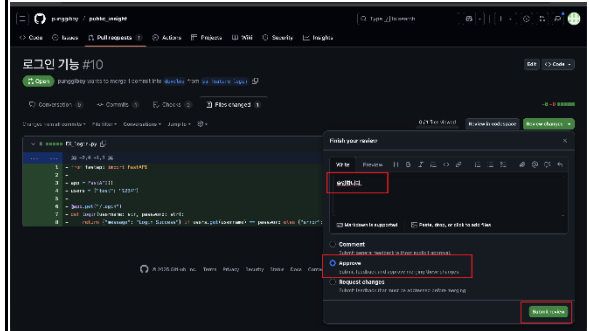
(4)ha의 깃허브



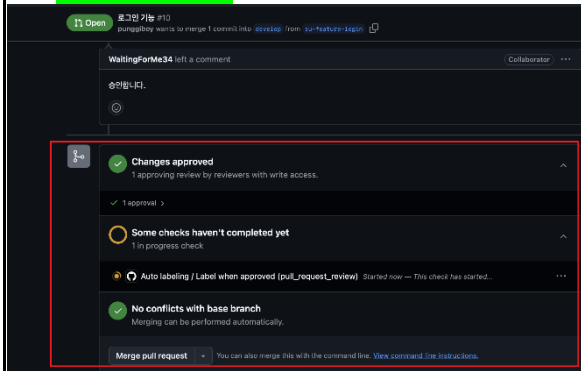
(5)ha의 깃허브



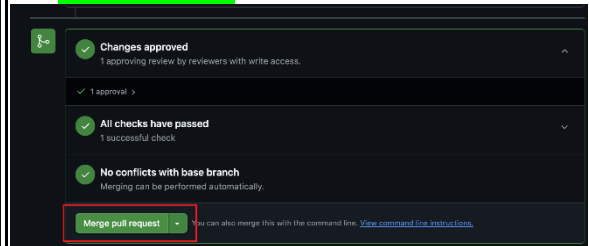
(6)ha의 깃허브



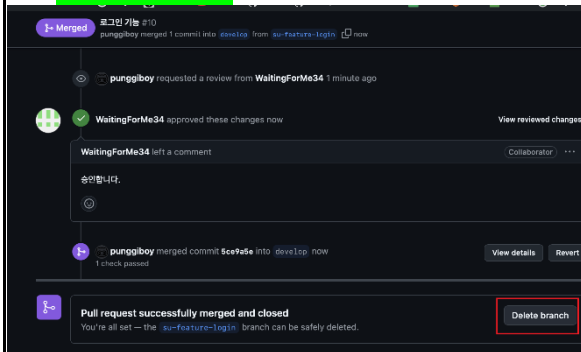
(7)su의 깃허브



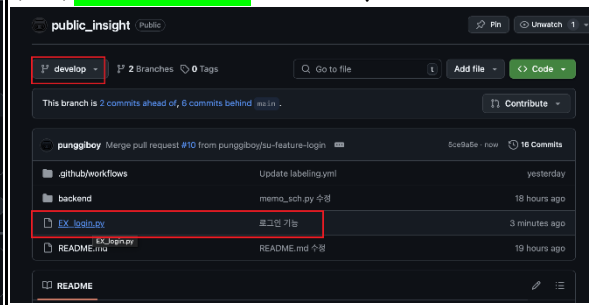
(8)su의 깃허브



(9)su의 깃허브



(10)su의 깃허브-develop 브랜치 확인



[3. Main 브랜치로 PR 요청 및 최종 배포]

develop 브랜치에서 테스트 진행

- develop 브랜치에 병합된 기능을 테스트
- 테스트가 통과하면 main 브랜치로 병합을 진행

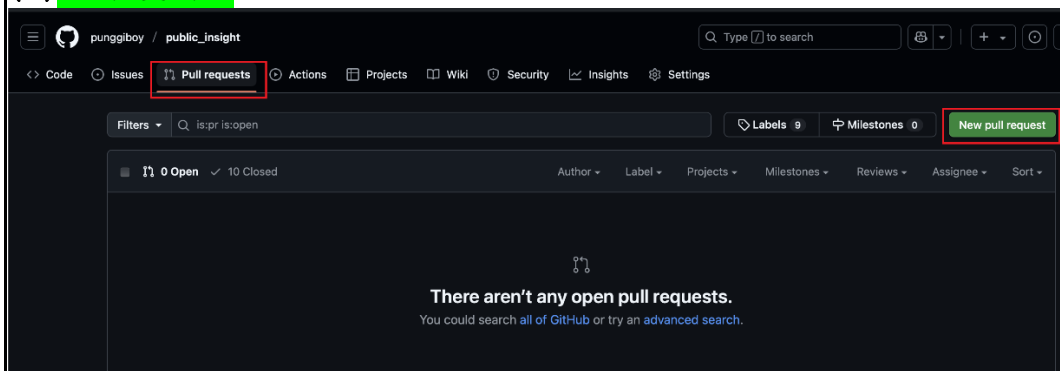
main 브랜치로 PR 요청

- GitHub에서 PR을 생성하여 병합 대상 브랜치를 main으로 설정
 - main <- develop
- 코드 리뷰 및 승인 담당자(예:ha)를 지정

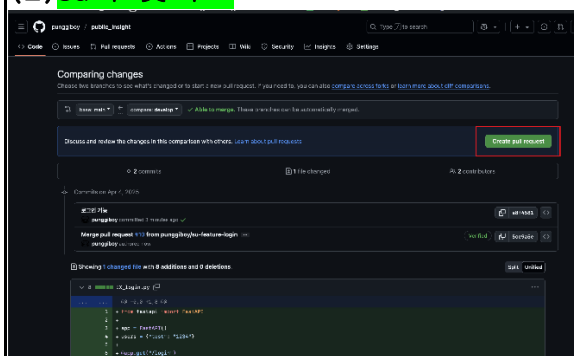
코드 리뷰 및 승인

- ha가 최종 검토 후 승인(Approve)함
- develop 브랜치의 변경 사항이 main 브랜치에 병합

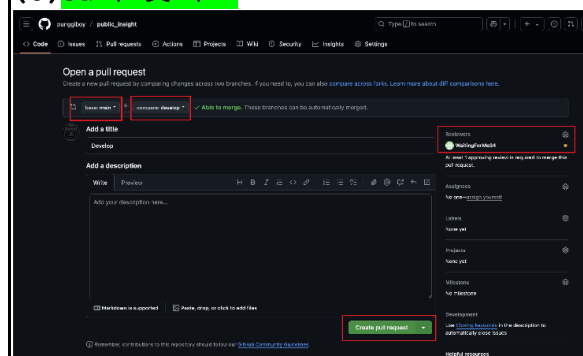
(1)su의 깃허브



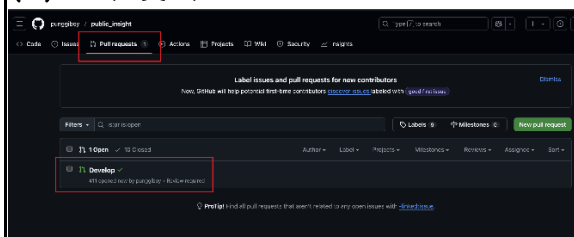
(2)su의 깃허브



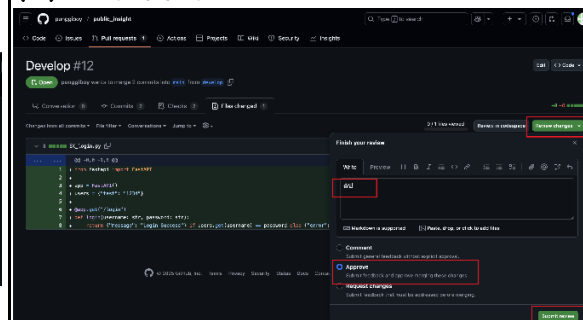
(3)su의 깃허브



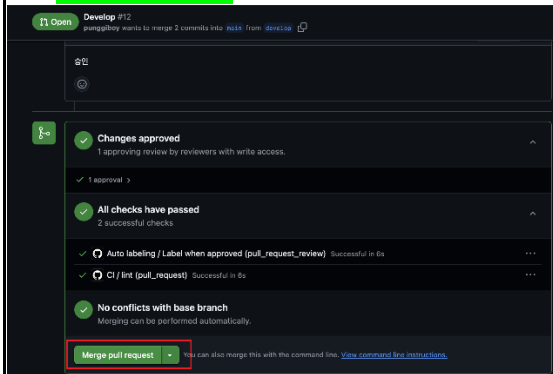
(4)ha의 깃허브



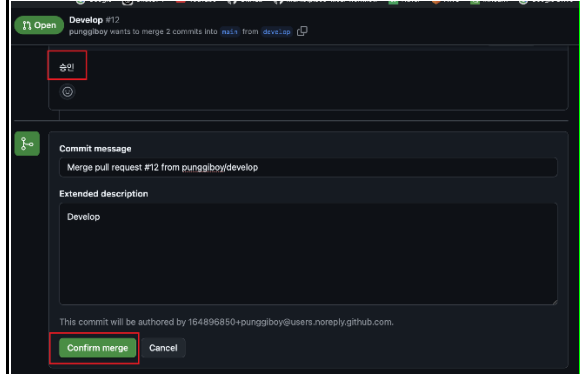
(5)ha의 깃허브



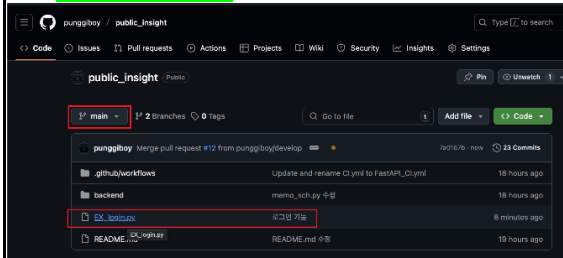
(6) su의 깃허브



(7) su의 깃허브



(8) su의 깃허브



[정리: Git 협업 프로세스 요약]

ha와 su는 역할이 언제든지 변할 수 있음

1. su는 develop 브랜치를 최신화한 후 새로운 기능 브랜치(su-feature-login)를 생성
2. 기능 개발 및 커밋 후 PR을 생성하여 develop 브랜치로 병합 요청
3. ha가 코드 리뷰 및 승인 후, su 또는 ha가 develop에 병합
4. develop 브랜치에서 테스트 후 main 브랜치로 PR 생성
5. ha가 최종 승인하여 main 브랜치에 병합(배포 가능 상태)